

```
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter_localizations/flutter_localizations.dart';

import 'l10n/app_localizations.dart';
import 'ui/measure_page.dart';
import 'ui/privacy.dart';

void main() {
  runApp(const CardCenteringApp());
}

class CardCenteringApp extends StatelessWidget {
  const CardCenteringApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      onGenerateTitle: (context) => AppLocalizations.of(context).appTitle,
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
        // 手机上用系统字体。
        fontFamily: kIsWeb ? 'NotoSansSCSubset' : null,
      ),
      localizationsDelegates: const [
        AppLocalizations.delegate,
        GlobalMaterialLocalizations.delegate,
        GlobalWidgetsLocalizations.delegate,
        GlobalCupertinoLocalizations.delegate,
      ],
      supportedLocales: AppLocalizations.supportedLocales,
      // 主要面向国内市场：只有英文系统显示英文，其他语言一律显示中文。
      localeResolutionCallback: (locale, supported) =>
        locale?.languageCode == 'en'
          ? const Locale('en')
          : const Locale('zh'),
      home: const PrivacyGate(child: MeasurePage()),
    );
  }
}

/// 参考线编号。前 4 条是竖线（x 坐标），后 4 条是横线（y 坐标），
/// 各自按从左到右、从上到下排列。
enum LineId {
  outerLeft,
  innerLeft,
  innerRight,
  outerRight,
  outerTop,
  innerTop,
  innerBottom,
}
```

```

outerBottom;

bool get isVertical => index < 4;

/// 在同方向 4 条线中的位置, 0-3。
int get order => index % 4;

bool get isOuter => order == 0 || order == 3;

/// 是否为左侧或上侧的线（否则为右侧或下侧）。
bool get isStartSide => order < 2;
}

/// 相邻两线之间的最小间距（原图像素）。
const double minLineGap = 1;

/// 8 条参考线的位置, 全部为原图像素坐标; x 为竖线, y 为横线。
class Guidelines {
    Guidelines({
        required this.outerLeft,
        required this.innerLeft,
        required this.innerRight,
        required this.outerRight,
        required this.outerTop,
        required this.innerTop,
        required this.innerBottom,
        required this.outerBottom,
    });

    /// 默认位置: 外 8%、内 14%, 右/下侧对称 (W、H 为原图宽高)。
    factory Guidelines.defaults(double width, double height) => Guidelines(
        outerLeft: 0.08 * width,
        innerLeft: 0.14 * width,
        innerRight: 0.86 * width,
        outerRight: 0.92 * width,
        outerTop: 0.08 * height,
        innerTop: 0.14 * height,
        innerBottom: 0.86 * height,
        outerBottom: 0.92 * height,
    );

    factory Guidelines._fromList(List<double> v) => Guidelines(
        outerLeft: v[0],
        innerLeft: v[1],
        innerRight: v[2],
        outerRight: v[3],
        outerTop: v[4],
        innerTop: v[5],
        innerBottom: v[6],
        outerBottom: v[7],
    );
}

```

```

double outerLeft, innerLeft, innerRight, outerRight;
double outerTop, innerTop, innerBottom, outerBottom;

List<double> _toList() => [
    outerLeft,
    innerLeft,
    innerRight,
    outerRight,
    outerTop,
    innerTop,
    innerBottom,
    outerBottom,
];

double operator [](LineId id) => _toList()[id.index];

Guidelines copy() => Guidelines._fromList(_toList());

/// 返回把 [id] 移到 [value] 后的新参考线。位置会被夹在相邻两线之间
/// （至少相隔 [minLineGap]），最外侧的线不超出图片 [width]×[height]。
Guidelines withLine(
    LineId id,
    double value, {
    required double width,
    required double height,
}) {
    final v = _toList();
    final base = id.isVertical ? 0 : 4;
    final limit = id.isVertical ? width : height;
    final o = id.order;
    final lo = o == 0 ? 0.0 : v[base + o - 1] + minLineGap;
    final hi = o == 3 ? limit : v[base + o + 1] - minLineGap;
    v[id.index] = value < lo ? lo : (value > hi ? hi : value);
    return Guidelines._fromList(v);
}
}

import 'dart:math' as math;

import '../models/guide_lines.dart';
import 'psa_table.dart';

class CenteringResult {
    const CenteringResult({
        required this.lrBig,
        required this.lrSmall,
        required this.tbBig,
        required this.tbSmall,
        required this.worse,
        required this.grade,
        required this.borderline,
        required this.nearGrade,
    });
}

```

```

});

final int lrBig, lrSmall; // 如 58 / 42
final int tbBig, tbSmall;
final int worse; // max(lrBig, tbBig)
final double? grade; // 10、9 ... 3; null 表示低于标准
final bool borderline; // 是否临界
final double? nearGrade; // 临界时接近的更高等级
}

/// 较宽一侧所占百分比, 四舍五入 (0.5 向上)。
int bigPercent(double a, double b) {
    final sum = a + b;
    if (sum <= 0) return 50;
    // 加一个极小量, 避免 57.5 这类值因浮点误差被舍成 57。
    return (100 * math.max(a, b) / sum + 0.5 + 1e-9).floor();
}

CenteringResult computeCentering(GuideLines g) {
    final l = g.innerLeft - g.outerLeft;
    final r = g.outerRight - g.innerRight;
    final t = g.innerTop - g.outerTop;
    final b = g.outerBottom - g.innerBottom;

    final lrBig = bigPercent(l, r);
    final tbBig = bigPercent(t, b);
    final worse = math.max(lrBig, tbBig);

    double? grade;
    PsaThreshold? higher; // 比所得等级高一级的阈值
    for (final th in psaFrontThresholds) {
        if (worse <= th.maxWorse) {
            grade = th.grade;
            break;
        }
        higher = th;
    }

    var borderline = false;
    double? nearGrade;
    if (higher != null && higher.grade >= borderlineMinGrade) {
        final over = worse - higher.maxWorse;
        if (over >= borderlineMinOver && over <= borderlineMaxOver) {
            borderline = true;
            nearGrade = higher.grade;
        }
    }

    return CenteringResult(
        lrBig: lrBig,
        lrSmall: 100 - lrBig,
        tbBig: tbBig,

```

```

        tbSmall: 100 - tbBig,
        worse: worse,
        grade: grade,
        borderline: borderline,
        nearGrade: nearGrade,
    );
}

import 'package:flutter/foundation.dart';
import 'package:shared_preferences/shared_preferences.dart';

/// 隐私政策版本。政策有实质变化时加 1，已同意旧版的用户会再次看到提示。
const int privacyPolicyVersion = 1;

const String _agreedKey = 'privacy_agreed_version';

/// 用户是否已同意当前版本的隐私政策。读取失败时按未同意处理，宁可多弹一次。
Future<bool> hasAgreedToPrivacyPolicy() async {
    try {
        final prefs = await SharedPreferences.getInstance();
        return (prefs.getInt(_agreedKey) ?? 0) >= privacyPolicyVersion;
    } catch (e) {
        debugPrint('Failed to read privacy consent: $e');
        return false;
    }
}

/// 记下用户已同意当前版本。只存一个版本号，不含个人信息。
Future<void> recordPrivacyAgreement() async {
    try {
        final prefs = await SharedPreferences.getInstance();
        await prefs.setInt(_agreedKey, privacyPolicyVersion);
    } catch (e) {
        debugPrint('Failed to save privacy consent: $e');
    }
}

/// PSA 正面居中阈值表（参考 PSA 公开标准）。
///
/// 所有与 PSA 标准相关的数字只允许出现在本文件中。
library;

class PsaThreshold {
    const PsaThreshold(this.grade, this.maxWorse);

    /// 等级，如 10、9 ... 3。
    final double grade;

    /// 该等级允许的较大一侧最大百分比（如 55 表示 55/45）。
    final int maxWorse;
}

```

```

/// 正面阈值，按等级从高到低排列。
const List<PsaThreshold> psaFrontThresholds = [
  PsaThreshold(10, 55),
  PsaThreshold(9, 60),
  PsaThreshold(8, 65),
  PsaThreshold(7, 70),
  PsaThreshold(6, 80),
  PsaThreshold(5, 85),
  PsaThreshold(3, 90),
];

/// 临界判断：超出上一级阈值的最小、最大百分点。
const int borderlineMinOver = 1;
const int borderlineMaxOver = 3;

/// 只有不低于此等级的“上一级”才做临界提示（PSA 对 7-10 级正面约有 5% 宽限）。
const double borderlineMinGrade = 7;

import 'package:flutter/material.dart';

import '../l10n/app_localizations.dart';

/// 旋转刻度盘：左右拖动刻度尺调整角度（-180°-180°），左侧显示当前角度，
/// 右侧“归零”“完成”。高度与微调栏相同，切换时画布不跳动。
class AngleDial extends StatelessWidget {
  const AngleDial({
    super.key,
    required this.angle,
    required this.onChanged,
    required this.onDone,
  });

  /// 当前角度（度），顺时针为正。
  final double angle;
  final ValueChanged<double> onChanged;
  final VoidCallback onDone;

  static const double maxAngle = 180;

  /// 每度对应的刻度尺像素；拖 8 点转 1°，0.8 点即 0.1°。
  static const double pixelsPerDegree = 8;

  @override
  Widget build(BuildContext context) {
    final l10n = AppLocalizations.of(context);
    final theme = Theme.of(context);
    return Row(
      children: [
        SizedBox(
          width: 64,
          child: Text(
            '${angle.toStringAsFixed(1)}°',

```

```

        key: const Key('rotate-angle'),
        style: theme.textTheme.titleMedium?.copyWith(
          fontWeight: FontWeight.w700,
          fontFeatures: const [FontFeature.tabularFigures()],
        ),
      ),
    ),
  ),
  Expanded(
    child: GestureDetector(
      key: const Key('rotate-dial'),
      behavior: HitTestBehavior.opaque,
      // 向左拖刻度尺, 角度增加 (和转动旋钮的手感一致)。
      onHorizontalDragUpdate: (d) => onChanged(
        (angle - d.delta.dx / pixelsPerDegree).clamp(-maxAngle, maxAngle),
      ),
      child: CustomPaint(
        size: const Size.fromHeight(52),
        painter: _DialPainter(
          angle: angle,
          tick: theme.colorScheme.onSurfaceVariant,
          accent: theme.colorScheme.primary,
          labelStyle: DefaultTextStyle.of(context).style.copyWith(
            fontSize: 9,
            color: theme.colorScheme.onSurfaceVariant,
          ),
        ),
      ),
    ),
  ),
  ),
  ),
  ),
  IconButton(
    key: const Key('rotate-reset'),
    tooltip: l10n.rotateReset,
    icon: const Icon(Icons.restart_alt),
    onPressed: angle == 0 ? null : () => onChanged(0),
  ),
  IconButton.filledTonal(
    key: const Key('rotate-done'),
    tooltip: l10n.done,
    icon: const Icon(Icons.check),
    onPressed: onDone,
  ),
],
);
}
}

```

```

class _DialPainter extends CustomPainter {
  _DialPainter({
    required this.angle,
    required this.tick,
    required this.accent,
    required this.labelStyle,

```

```

});

final double angle;
final Color tick, accent;
final TextStyle labelStyle;

@override
void paint(Canvas canvas, Size size) {
  canvas.clipRect(Offset.zero & size);
  final cx = size.width / 2;
  const ppd = AngleDial.pixelsPerDegree;
  final half = size.width / 2 / ppd;
  final paint = Paint()
    ..color = tick
    ..strokeWidth = 1;
  final from = (angle - half).floor();
  final to = (angle + half).ceil();
  for (var deg = from; deg <= to; deg++) {
    if (deg.abs() > AngleDial.maxAngle) continue;
    final x = cx + (deg - angle) * ppd;
    final major = deg % 10 == 0;
    final len = major ? 16.0 : (deg % 5 == 0 ? 11.0 : 6.0);
    canvas.drawLine(Offset(x, 30 - len), Offset(x, 30), paint);
    if (major) {
      final t = TextPainter(
        text: TextSpan(text: '$deg', style: labelStyle),
        textDirection: TextDirection.ltr,
      )..layout();
      t.paint(canvas, Offset(x - t.width / 2, 34));
      t.dispose();
    }
  }
  // 中间的指针
  canvas.drawLine(
    Offset(cx, 8),
    Offset(cx, 32),
    Paint()
      ..color = accent
      ..strokeWidth = 2.5
      ..strokeCap = StrokeCap.round,
  );
}

@override
bool shouldRepaint(_DialPainter old) =>
  old.angle != angle || old.tick != tick || old.accent != accent;
}

import 'package:flutter/widgets.dart';

import '../models/guide_lines.dart';

```

```

/// 画布几何：图片在视口中按比例放到最大并居中显示；
/// 负责原图像素坐标与屏幕坐标之间的换算，以及手柄标签的摆放。
@immutable
class CanvasGeometry {
  factory CanvasGeometry({
    required Size viewport,
    required double imageWidth,
    required double imageHeight,
    Matrix4? transform,
  }) {
    final avail = Offset.zero & viewport;
    var fit = avail.width / imageWidth;
    if (avail.height / imageHeight < fit) fit = avail.height / imageHeight;
    if (fit < 0) fit = 0;
    final rect = Rect.fromCenter(
      center: avail.center,
      width: imageWidth * fit,
      height: imageHeight * fit,
    );
    final m = transform ?? Matrix4.identity();
    return CanvasGeometry._(
      viewport,
      fit,
      rect,
      m.getMaxScaleOnAxis(),
      Offset(m.storage[12], m.storage[13]),
    );
  }

  const CanvasGeometry._(
    this.viewport,
    this.fitScale,
    this.imageRect,
    this.zoom,
    this.pan,
  );

  final Size viewport;

  /// 1 原图像素在未缩放画布上的逻辑像素数。
  final double fitScale;

  /// 图片在未缩放画布上的位置。
  final Rect imageRect;

  /// 当前缩放倍数与平移量（来自画布的变换矩阵，只含缩放和平移）。
  final double zoom;
  final Offset pan;

  /// 1 原图像素在屏幕上的逻辑像素数。
  double get screenPerPixel => zoom * fitScale;

```

```

double xToScreen(double px) =>
    zoom * (imageRect.left + px * fitScale) + pan.dx;

double yToScreen(double py) =>
    zoom * (imageRect.top + py * fitScale) + pan.dy;

/// 屏幕上的位移换算成原图像素。
double screenToImage(double delta) =>
    screenPerPixel == 0 ? 0 : delta / screenPerPixel;

/// 图片在屏幕上的位置（可能超出视口）。
Rect get screenImageRect => Rect.fromLTRB(
    xToScreen(0),
    yToScreen(0),
    zoom * imageRect.right + pan.dx,
    zoom * imageRect.bottom + pan.dy,
);

/// 手柄标签的大小：上下两边的标签横放（图示在左、文字在右），
/// 左右两边的标签竖放（图示在上、文字在下）。
static const Size horizontalLineHandleSize = Size(76, 36);
static const Size verticalLineHandleSize = Size(44, 60);

/// 手柄标签在屏幕上的区域，也是它的触摸区域。
///
/// 每条边的两个标签放在这条边的中间：外框标签在线的外侧，内框标签
/// 在线的内侧，中间隔着边框，不会重叠。放大后标签沿线移动，停在屏幕上
/// 可见的那一段，并且整个标签保持在视口内。
Rect handleRect(LineId id, Guidelines lines) {
    const gap = 2.0;
    final visible = screenImageRect.intersect(Offset.zero & viewport);
    // 朝外：上边的外框线往上、下边的外框线往下；内框线相反。
    final outward = id.isStartSide == id.isOuter;
    Rect r;
    if (id.isVertical) {
        final size = verticalLineHandleSize;
        final x = xToScreen(lines[id]);
        final mid =
            (yToScreen(lines.outerTop) + yToScreen(lines.outerBottom)) / 2;
        final y = _clampInto(mid, visible.top, visible.bottom, size.height);
        final left = outward ? x - gap - size.width : x + gap;
        r = Rect.fromLTWH(left, y - size.height / 2, size.width, size.height);
    } else {
        final size = horizontalLineHandleSize;
        final y = yToScreen(lines[id]);
        final mid =
            (xToScreen(lines.outerLeft) + xToScreen(lines.outerRight)) / 2;
        final x = _clampInto(mid, visible.left, visible.right, size.width);
        final top = outward ? y - gap - size.height : y + gap;
        r = Rect.fromLTWH(x - size.width / 2, top, size.width, size.height);
    }
}
// 整个标签保持在视口内（卡片贴着照片边缘时，外框标签会被推回来）。

```

```

    final dx = r.left < 0
      ? -r.left
      : (r.right > viewport.width ? viewport.width - r.right : 0.0);
    final dy = r.top < 0
      ? -r.top
      : (r.bottom > viewport.height ? viewport.height - r.bottom : 0.0);
    return r.shift(Offset(dx, dy));
  }

  /// 标签中心。
  Offset handleCenter(LineId id, GuideLines lines) =>
    handleRect(id, lines).center;

  /// 把长度为 [extent] 的标签中心 [v] 夹在 [lo, hi] 区间内；区间放不下时居中。
  static double _clampInto(double v, double lo, double hi, double extent) {
    final a = lo + extent / 2, b = hi - extent / 2;
    if (a > b) return (lo + hi) / 2;
    return _clamp(v, a, b);
  }

  static double _clamp(double v, double lo, double hi) =>
    v < lo ? lo : (v > hi ? hi : v);

  @override
  bool operator ==(Object other) =>
    other is CanvasGeometry &&
    other.viewport == viewport &&
    other.fitScale == fitScale &&
    other.imageRect == imageRect &&
    other.zoom == zoom &&
    other.pan == pan;

  @override
  int get hashCode => Object.hash(viewport, fitScale, imageRect, zoom, pan);
}

import 'package:flutter/material.dart';

import '../models/guide_lines.dart';
import 'canvas_geometry.dart';

const Color outerLineColor = Color(0xFF00E5FF);
const Color innerLineColor = Color(0xFFFFF4081);
const Color selectedLineColor = Color(0xFFFFFEA00);

/// 在图片上方绘制 8 条参考线和它们的手柄。
///
/// 外框线与内框线用三重方式区分：颜色（青 / 粉）、线型（长虚线 / 短虚线）、
/// 手柄标签（双框小图示高亮要对准的那条边，旁边写“外左”“内上”等）。
class GuidePainter extends CustomPainter {
  GuidePainter({
    required this.geometry,

```

```

        required this.lines,
        required this.selected,
        required this.labelOf,
        required this.labelStyle,
        this.showHandles = true,
    });

    final CanvasGeometry geometry;
    final GuideLines lines;
    final LineId? selected;

    /// 手柄上的文字, 如“外左”“内上”。
    final String Function(LineId id) labelOf;
    final TextStyle labelStyle;

    /// 是否画手柄标签。拖线时隐藏, 免得标签被放大镜放大、挡住卡片边缘。
    final bool showHandles;

    /// 外框线长虚线、内框线短虚线: 空隙露出底下的卡片边缘, 不会把边缘挡住。
    static const double _outerDash = 14, _outerGap = 6;
    static const double _innerDash = 5, _innerGap = 5;

    static Color colorOf(LineId id, {required bool selected}) => selected
        ? selectedLineColor
        : (id.isOuter ? outerLineColor : innerLineColor);

    @override
    void paint(Canvas canvas, Size size) {
        canvas.clipRect(Offset.zero & size);
        final img = geometry.screenImageRect;

        // 选中的线最后画, 压在其他线上面。
        final order = [
            for (final id in LineId.values)
                if (id != selected) id,
            if (selected != null) selected!,
        ];
        for (final id in order) {
            final isSel = id == selected;
            final color = colorOf(id, selected: isSel);
            final handle = geometry.handleCenter(id, lines);
            final line = Paint()
                ..color = color
                ..strokeWidth = isSel ? 2.5 : 1.5;
            // 线贯穿图片。外框线长虚线, 内框线短虚线。
            if (id.isVertical) {
                final x = geometry.xToScreen(lines[id]);
                _drawLine(canvas, id.isOuter, true, x, img.top, img.bottom, line);
            } else {
                final y = geometry.yToScreen(lines[id]);
                _drawLine(canvas, id.isOuter, false, y, img.left, img.right, line);
            }
        }
    }

```

```

        if (showHandles) _paintHandle(canvas, handle, id, color, isSel);
    }
}

```

/// 画一条水平或竖直的虚线。[at] 是线所在的 x（竖线）或 y（横线），  
 /// [from]-[to] 是线的另一方向的范围。外框线长虚线、内框线短虚线；  
 /// 虚线的起点对齐到屏幕坐标，拖动时纹路不会跟着滑动。

```

void _drawLine(
    Canvas canvas,
    bool outer,
    bool vertical,
    double at,
    double from,
    double to,
    Paint paint,
) {
    Offset p(double t) => vertical ? Offset(at, t) : Offset(t, at);
    final dash = outer ? _outerDash : _innerDash;
    final period = dash + (outer ? _outerGap : _innerGap);
    var start = (from / period).floorToDouble() * period;
    for (; start < to; start += period) {
        final a = start < from ? from : start;
        final b = start + dash > to ? to : start + dash;
        if (b > a) canvas.drawLine(p(a), p(b), paint);
    }
}

```

```

void _paintHandle(
    Canvas canvas,
    Offset c,
    LineId id,
    Color color,
    bool isSel,
) {
    final rect = geometry.handleRect(id, lines);
    final rr = RRect.fromRectAndRadius(rect, const Radius.circular(10));
    canvas.drawRRect(rr, Paint()..color = const Color(0xD91C1E));
    if (isSel) {
        canvas.drawRRect(
            rr.deflate(1),
            Paint()
                ..color = color
                ..style = PaintingStyle.stroke
                ..strokeWidth = 2,
        );
    }
}

final text = TextPainter(
    text: TextSpan(
        text: labelOf(id),
        style: labelStyle.copyWith(
            color: color,
            fontSize: 13,

```

```

        fontWeight: FontWeight.w700,
        height: 1,
    ),
),
textDirection: TextDirection.ltr,
)..layout();
// 左右两边的标签竖放：图示在上、文字在下；上下两边的标签横放：图示在左、文字在右。
final Offset glyphAt, textAt;
if (id.isVertical) {
    glyphAt = Offset(rect.center.dx, rect.top + 19);
    textAt = Offset(rect.center.dx, rect.bottom - 13);
} else {
    glyphAt = Offset(rect.left + 18, rect.center.dy);
    textAt = Offset(rect.left + 50, rect.center.dy);
}
_paintGlyph(canvas, glyphAt, id, color);
text.paint(canvas, textAt - Offset(text.width / 2, text.height / 2));
text.dispose();
}

```

/// 双框小卡片：外框套内框，用线的颜色高亮这条线要对准的那一条边。

```

void _paintGlyph(Canvas canvas, Offset c, LineId id, Color color) {
    final outer = Rect.fromCenter(center: c, width: 18, height: 25);
    final inner = Rect.fromCenter(center: c, width: 10, height: 15);
    final stroke = Paint()
        ..color = const Color(0xFFC8C8CD)
        ..style = PaintingStyle.stroke
        ..strokeWidth = 1.2;
    canvas.drawRect(outer, stroke);
    canvas.drawRect(inner, stroke);
    final r = id.isOuter ? outer : inner;
    final edge = switch (id) {
        LineId.outerLeft || LineId.innerLeft => [r.topLeft, r.bottomLeft],
        LineId.outerRight || LineId.innerRight => [r.topRight, r.bottomRight],
        LineId.outerTop || LineId.innerTop => [r.topLeft, r.topRight],
        LineId.outerBottom || LineId.innerBottom => [r.bottomLeft, r.bottomRight],
    };
    canvas.drawLine(
        edge[0],
        edge[1],
        Paint()
            ..color = color
            ..strokeWidth = 3
            ..strokeCap = StrokeCap.square,
    );
}

```

@override

```

bool shouldRepaint(GuidePainter oldDelegate) =>
    oldDelegate.lines != lines ||
    oldDelegate.selected != selected ||
    oldDelegate.geometry != geometry ||

```

```

        oldDelegate.labelStyle != labelStyle ||
        oldDelegate.showHandles != showHandles;
    }

import 'dart:math' as math;
import 'dart:ui' as ui;

import 'package:flutter/foundation.dart';

/// 超过这个像素数的图片只解码一张缩小的显示图。
const int maxDisplayPixels = 12000000;

/// 已解码、可显示的图片。
class LoadedImage {
  LoadedImage(this.image, this.width, this.height);

  /// 显示用图片，可能比原图小。
  final ui.Image image;

  /// 原图尺寸（已按 EXIF 摆正），参考线坐标以此为准。
  final double width, height;

  /// 显示图每像素对应的原图像素数。
  double get displayToOriginal => width / image.width;

  void dispose() => image.dispose();
}

/// 解码图片。解码由引擎在后台线程完成，并按 EXIF 摆正方向；
/// 超过 [maxPixels] 的图直接按比例解码成缩略图，避免占用大量内存。
Future<LoadedImage> decodeForDisplay(
  Uint8List bytes, {
    int maxPixels = maxDisplayPixels,
  }) async {
  int? w, h;
  final ui.Codec codec;
  if (kIsWeb) {
    // 网页端解码前拿不到原图尺寸，交给浏览器完整解码。
    codec = await ui.instantiateImageCodec(bytes);
  } else {
    // 注意：不能自己创建 ImageDescriptor 并在取帧前释放它—解码发生在
    // getNextFrame 时，提前释放会导致引擎解码线程崩溃。这里交给引擎管理，
    // buffer 也由它负责释放。
    final buffer = await ui.ImmutableBuffer.fromUint8List(bytes);
    codec = await ui.instantiateImageCodecWithSize(
      buffer,
      getTargetSize: (iw, ih) {
        w = iw;
        h = ih;
        if (iw * ih <= maxPixels) return const ui.TargetImageSize();
        return ui.TargetImageSize(
          width: (iw * math.sqrt(maxPixels / (iw * ih))).round(),

```

```

        );
    },
    );
}
final ui.FrameInfo frame;
try {
    frame = await codec.getNextFrame();
} finally {
    codec.dispose();
}
final image = frame.image;

var ow = (w ?? image.width).toDouble();
var oh = (h ?? image.height).toDouble();
// 引擎给的是未旋转的尺寸；EXIF 旋转 90° 的图解码后宽高互换。
if ((image.width > image.height) != (ow > oh) &&
    image.width != image.height) {
    final t = ow;
    ow = oh;
    oh = t;
}
return LoadedImage(image, ow, oh);
}

```

```
import 'dart:math' as math;
```

```
import 'package:flutter/gestures.dart';
import 'package:flutter/material.dart';
import 'package:image_picker/image_picker.dart';
```

```
import '../l10n/app_localizations.dart';
import '../logic/centering.dart';
import '../models/guide_lines.dart';
import 'angle_dial.dart';
import 'canvas_geometry.dart';
import 'guidePainter.dart';
import 'image_loader.dart';
import 'picked_file_cleanup.dart';
import 'privacy.dart';
import 'result_bar.dart';
```

/// 唯一的页面：空状态，或图片 + 参考线 + 结果栏。

```
class MeasurePage extends StatefulWidget {
    const MeasurePage({super.key});

    @override
    State<MeasurePage> createState() => _MeasurePageState();
}
```

```
class _MeasurePageState extends State<MeasurePage> {
    LoadedImage? _image;
    bool _loading = false;
```

```
@Override
void dispose() {
    _image?.dispose();
    super.dispose();
}

Future<void> _pick() async {
    final l10n = AppLocalizations.of(context);
    final messenger = ScaffoldMessenger.of(context);
    final XFile? file;
    try {
        // requestFullMetadata: false — iOS 上不需要相册访问权限。
        file = await ImagePicker().pickImage(
            source: ImageSource.gallery,
            requestFullMetadata: false,
        );
    } catch (e) {
        debugPrint('Photo picker failed: $e');
        messenger.showSnackBar(SnackBar(content: Text(l10n.imageLoadFailed)));
        return;
    }
    if (file == null) return;

    setState(() => _loading = true);
    LoadedImage? loaded;
    try {
        final bytes = await file.readAsBytes();
        await deletePickedCopy(file.path);
        loaded = await decodeForDisplay(bytes);
    } catch (e) {
        // 只记录错误类型与信息, 不含图片内容。
        debugPrint('Failed to load picked photo: $e');
        loaded = null;
    }
    if (!mounted) {
        loaded?.dispose();
        return;
    }
    setState(() {
        _loading = false;
        if (loaded != null) {
            _image?.dispose();
            _image = loaded;
        }
    });
    if (loaded == null) {
        messenger.showSnackBar(SnackBar(content: Text(l10n.imageLoadFailed)));
    }
}

void _showAbout() {
```

```

final l10n = AppLocalizations.of(context);
showDialog<void>(
  context: context,
  builder: (context) => AlertDialog(
    title: Text(l10n.about),
    content: SingleChildScrollView(child: Text(l10n.aboutBody)),
    actions: [
      TextButton(
        onPressed: () {
          Navigator.of(context).pop();
          showPrivacyPolicy(this.context);
        },
        child: Text(l10n.privacyPolicy),
      ),
      TextButton(
        onPressed: () => Navigator.of(context).pop(),
        child: Text(l10n.close),
      ),
    ],
  ),
);
}

```

```

@override
Widget build(BuildContext context) {
  final l10n = AppLocalizations.of(context);
  final image = _image;
  return Scaffold(
    appBar: AppBar(
      title: Text(l10n.appTitle),
      actions: [
        if (image != null)
          IconButton(
            tooltip: l10n.changePhoto,
            icon: const Icon(Icons.photo_library_outlined),
            onPressed: _loading ? null : _pick,
          ),
          IconButton(
            tooltip: l10n.about,
            icon: const Icon(Icons.info_outline),
            onPressed: _showAbout,
          ),
      ],
    ),
    body: _loading
      ? const Center(child: CircularProgressIndicator())
      : image == null
        ? _EmptyState(onPick: _pick)
        : MeasureView(key: ObjectKey(image), image: image),
  );
}
}

```

```
class _EmptyState extends StatelessWidget {
  const _EmptyState({required this.onPick});

  final VoidCallback onPick;

  @override
  Widget build(BuildContext context) {
    final l10n = AppLocalizations.of(context);
    final theme = Theme.of(context);
    return Center(
      child: Padding(
        padding: const EdgeInsets.all(32),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            Icon(
              Icons.crop_portrait,
              size: 72,
              color: theme.colorScheme.primary,
            ),
            const SizedBox(height: 16),
            Text(
              l10n.emptyTitle,
              style: theme.textTheme.titleLarge,
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 8),
            Text(
              l10n.emptyHint,
              style: theme.textTheme.bodyMedium,
              textAlign: TextAlign.center,
            ),
            const SizedBox(height: 24),
            FilledButton.icon(
              onPressed: onPick,
              icon: const Icon(Icons.photo_library_outlined),
              label: Text(l10n.pickPhoto),
            ),
          ],
        ),
      ),
    );
  }
}
```

/// 图片测量区域：图片、参考线、结果栏。

```
class MeasureView extends StatefulWidget {
  const MeasureView({super.key, required this.image});

  final LoadedImage image;
```

```

@override
State<MeasureView> createState() => _MeasureViewState();
}

class _MeasureViewState extends State<MeasureView> {
  late final ValueNotifier<GuideLines> _lines = ValueNotifier(
    GuideLines.defaults(widget.image.width, widget.image.height),
  );
  final TransformationController _transform = TransformationController();
  final GlobalKey _canvasKey = GlobalKey();
  LineId? _selected;

  /// 图片旋转角度（度，顺时针为正）与是否显示刻度盘。参考线不随图片转，
  /// 用户把歪斜的卡片转正，对齐到水平、竖直的参考线上。
  double _angle = 0;
  bool _rotating = false;

  /// 最近一次布局的画布几何，拖动时用来换算坐标。
  CanvasGeometry? _geo;

  // 拖动中的状态：线的起始位置与手指按下的位置。线的位置始终由“手指
  // 离按下点多远”算出，而不是把每次的移动量累加：有的平台（如鸿蒙模拟器
  // 用鼠标时）会重复发送移动事件，累加会让线跑得比手指快、来回撞到边界。
  LineId? _dragging;
  double _dragStart = 0;
  Offset _dragOrigin = Offset.zero;

  /// 拖动时手指在画布上的位置，用于放大镜；null 表示不显示。
  Offset? _finger;

  static const double _magnifierSize = 110;
  static const double _magnifierLift = 80;
  static const double _magnification = 3;

  @override
  void dispose() {
    _lines.dispose();
    _transform.dispose();
    super.dispose();
  }

  void _moveLine(LineId id, double value) {
    _lines.value = _lines.value.withLine(
      id,
      value,
      width: widget.image.width,
      height: widget.image.height,
    );
  }

  Offset? _toCanvas(Offset global) {
    final box = _canvasKey.currentContext?.findRenderObject() as RenderBox?;

```

```
        return box?.globalToLocal(global);
    }

    void _onDragStart(LineId id, DragStartDetails d) {
        setState(() {
            _rotating = false;
            _selected = id;
            _dragging = id;
            _finger = _toCanvas(d.globalPosition);
        });
        _dragStart = _lines.value[id];
        _dragOrigin = d.globalPosition;
    }

    void _onDragUpdate(DragUpdateDetails d) {
        final id = _dragging;
        final geo = _geo;
        if (id == null || geo == null) return;
        final moved = d.globalPosition - _dragOrigin;
        // 屏幕位移 ÷ 当前缩放倍数, 换算成原图像素。
        _moveLine(
            id,
            _dragStart + geo.screenToImage(id.isVertical ? moved.dx : moved.dy),
        );
        setState(() => _finger = _toCanvas(d.globalPosition));
    }

    void _onDragEnd() {
        setState(() {
            _dragging = null;
            _finger = null;
        });
    }

    void _nudge(int direction) {
        final id = _selected;
        if (id == null) return;
        _moveLine(id, _lines.value[id] + direction);
    }

    @override
    Widget build(BuildContext context) {
        return Column(
            children: [
                Expanded(
                    child: LayoutBuilder(
                        builder: (context, constraints) {
                            final viewport = constraints.biggest;
                            final base = CanvasGeometry(
                                viewport: viewport,
                                imageWidth: widget.image.width,
                                imageHeight: widget.image.height,
```

```

);
return ClipRect(
  child: ColoredBox(
    color: Colors.black,
    child: Stack(
      key: _canvasKey,
      children: [
        Positioned.fill(
          child: InteractiveViewer(
            transformationController: _transform,
            minScale: 1,
            maxScale: 8,
            child: GestureDetector(
              behavior: HitTestBehavior.opaque,
              onTap: () => setState(() => _selected = null),
              child: SizedBox.fromSize(
                size: viewport,
                child: Stack(
                  children: [
                    Positioned.fromRect(
                      rect: base.imageRect,
                      // 绕图片中心旋转，超出原图范围的部分裁掉。
                      child: ClipRect(
                        child: Transform.rotate(
                          key: const Key('rotated-image'),
                          angle: _angle * math.pi / 180,
                          child: RawImage(
                            image: widget.image.image,
                            fit: BoxFit.fill,
                            filterQuality: FilterQuality.medium,
                          ),
                        ),
                      ),
                    ),
                  ],
                ),
              ),
            ),
          ),
        ),
      ],
    ),
  ),
  // 参考线与手柄画在缩放层之外，用同一个变换矩阵换算位置，
  // 这样线宽和手柄大小不随缩放变化，位置始终与图片对齐。
  Positioned.fill(
    child: AnimatedBuilder(
      animation: Listenable.merge([_transform, _lines]),
      builder: (context, _) {
        final geo = CanvasGeometry(
          viewport: viewport,
          imageWidth: widget.image.width,
          imageHeight: widget.image.height,
          transform: _transform.value,
        );
      },
    ),
  );

```

```

        _geo = geo;
        return Stack(children: _overlay(geo, _lines.value));
      },
    ),
  ),
],
),
),
);
},
),
),
),
);
},
),
),
_nudgeBar(_selected),
ValueListenableBuilder<Guidelines>(
  valueListenable: _lines,
  builder: (context, lines, _) =>
    ResultBar(result: computeCentering(lines)),
),
],
);
}

```

/// 参考线绘制层、每个手柄标签一块触摸区域、拖动时的放大镜。

```

List<Widget> _overlay(CanvasGeometry geo, Guidelines lines) {
  // 选中的手柄放在最上层，重叠时优先响应。
  final ids = [
    for (final id in LineId.values)
      if (id != _selected) id,
    if (_selected != null) _selected!,
  ];
  final finger = _finger;
  final dragging = _dragging;
  return [
    Positioned.fill(
      child: IgnorePointer(
        child: CustomPaint(
          painter: GuidePainter(
            geometry: geo,
            lines: lines,
            selected: _selected,
            labelOf: (id) =>
              AppLocalizations.of(context).handleLabel(id.name),
            labelStyle: DefaultTextStyle.of(context).style,
            showHandles: _dragging == null,
          ),
        ),
      ),
    ),
  ],
  for (final id in ids)
    if (_onScreen(geo, geo.handleCenter(id, lines)))
      Positioned(
        // key 放在最外层：选中后手柄会调整层级，保持同一个手势不中断。

```

```

        key: ValueKey('handle-${id.name}'),
        left: geo.handleRect(id, lines).left,
        top: geo.handleRect(id, lines).top,
        width: geo.handleRect(id, lines).width,
        height: geo.handleRect(id, lines).height,
        child: GestureDetector(
          behavior: HitTestBehavior.opaque,
          dragStartBehavior: DragStartBehavior.down,
          onTap: () => setState(() {
            _rotating = false;
            _selected = id;
          }),
          onPanStart: (d) => _onDragStart(id, d),
          onPanUpdate: _onDragUpdate,
          onPanEnd: (_) => _onDragEnd(),
          onPanCancel: _onDragEnd,
        ),
      ),
    if (finger != null && dragging != null)
      ..._magnifier(geo, lines, dragging, finger),
  ];
}

```

/// 放大镜显示在手指上方 80 点（放不下时改到下方），放大被拖动的线上  
 /// 与手指对应的那一点：拖竖线时取手指的高度，拖横线时取手指的水平位置。  
 /// 手指沿线方向移动不会改变线的位置，只换放大的地方，可以沿线逐段检查。

```

List<Widget> _magnifier(
  CanvasGeometry geo,
  GuideLines lines,
  LineId id,
  Offset finger,
) {
  const r = _magnifierSize / 2;
  final vp = geo.viewport;
  var center = finger - const Offset(0, _magnifierLift);
  if (center.dy - r < 0) center = finger + const Offset(0, _magnifierLift);
  center = Offset(
    center.dx.clamp(r, vp.width - r).toDouble(),
    center.dy.clamp(r, vp.height - r).toDouble(),
  );

  final img = geo.screenImageRect.intersect(Offset.zero & vp);
  final Offset focus = id.isVertical
    ? Offset(
        geo.xToScreen(lines[id]),
        finger.dy.clamp(img.top, img.bottom).toDouble(),
      )
    : Offset(
        finger.dx.clamp(img.left, img.right).toDouble(),
        geo.yToScreen(lines[id]),
      );
  return [

```

```

Positioned(
  left: center.dx - r,
  top: center.dy - r,
  child: RawMagnifier(
    key: const Key('magnifier'),
    size: const Size.square(_magnifierSize),
    magnificationScale: _magnification,
    focalPointOffset: focus - center,
    decoration: MagnifierDecoration(
      shape: CircleBorder(
        side: BorderSide(
          color: GuidePainter.colorOf(id, selected: true),
          width: 3,
        ),
      ),
    ),
  ),
),
);
}

```

/// 微调栏始终占位，选中与否、显示刻度盘与否都不改变画布大小，避免图片跳动。

```

Widget _nudgeBar(LineId? id) {
  final l10n = AppLocalizations.of(context);
  final theme = Theme.of(context);
  return Material(
    color: theme.colorScheme.surface,
    child: Container(
      height: 52,
      padding: const EdgeInsets.symmetric(horizontal: 16),
      alignment: Alignment.centerLeft,
      child: _rotating
        ? AngleDial(
            angle: _angle,
            onChanged: (a) => setState(() => _angle = a),
            onDone: () => setState(() => _rotating = false),
          )
        : id == null
          ? Row(
              children: [
                Expanded(
                  child: Text(
                    l10n.nudgeHint,
                    style: theme.textTheme.bodyMedium?.copyWith(
                      color: theme.colorScheme.onSurfaceVariant,
                    ),
                  ),
                ),
              ],
            )
          : TextButton.icon(
              key: const Key('rotate-open'),
              onPressed: () => setState(() => _rotating = true),
              icon: const Icon(Icons.rotate_right),
            ),
    ),
  );
}

```

```

        label: Text(
          _angle == 0
            ? l10n.rotate
            : '${l10n.rotate} ${_angle.toStringAsFixed(1)}°',
        ),
      ),
    ],
  ),
  : Row(
    children: [
      Icon(
        Icons.circle,
        size: 12,
        color: GuidePainter.colorOf(id, selected: true),
      ),
      const SizedBox(width: 8),
      Expanded(
        child: Text(
          l10n.lineName(id.name),
          style: theme.textTheme.titleSmall,
        ),
      ),
      IconButton.filledTonal(
        key: const Key('nudge-minus'),
        tooltip: l10n.moveOnePixel,
        icon: Icon(
          id.isVertical ? Icons.arrow_back : Icons.arrow_upward,
        ),
        onPressed: () => _nudge(-1),
      ),
      const SizedBox(width: 12),
      IconButton.filledTonal(
        key: const Key('nudge-plus'),
        tooltip: l10n.moveOnePixel,
        icon: Icon(
          id.isVertical
            ? Icons.arrow_forward
            : Icons.arrow_downward,
        ),
        onPressed: () => _nudge(1),
      ),
    ],
  ),
),
);
}

```

```

static bool _onScreen(CanvasGeometry geo, Offset c) =>
  c.dx >= 0 &&
  c.dx <= geo.viewport.width &&
  c.dy >= 0 &&
  c.dy <= geo.viewport.height;

```

```

}

export 'picked_file_cleanup_stub.dart'
  if (dart.library.io) 'picked_file_cleanup_io.dart';

import 'dart:io';

/// 系统选图器会把所选图片复制到 App 的临时目录；读完字节后删掉这份副本，
/// 保证不在 App 目录里留下用户图片。只删临时目录下的文件，绝不碰原图。
Future<void> deletePickedCopy(String path) async {
  final p = path.replaceAll('\\', '/');
  if (!p.contains('/cache/') && !p.contains('/tmp/')) return;
  try {
    final file = File(path);
    await file.delete();
    // 安卓上 image_picker 会为每次选图建一个随机命名的子目录，一并删掉；
    // 非递归删除，目录不空时会失败而不会误删别的文件。
    final dir = file.parent;
    final name = dir.uri.pathSegments.where((s) => s.isNotEmpty).last;
    if (name != 'cache' && name != 'tmp') await dir.delete();
  } catch (_) {}
  // 删除失败不影响使用，系统会自行清理临时目录。
}

/// 网页端选图不产生临时文件，无需清理。
Future<void> deletePickedCopy(String path) async {}

import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import '../l10n/app_localizations.dart';
import '../logic/privacy_consent.dart';

/// 打开完整的隐私政策页面。
void showPrivacyPolicy(BuildContext context) {
  Navigator.of(context).push(
    MaterialPageRoute<void>(builder: (_) => const PrivacyPolicyPage()),
  );
}

/// 首次启动时弹出隐私政策提示，同意后才能使用。
///
/// 国内应用商店要求 App 首次启动时用弹窗告知隐私规则。网页版不弹：
/// 网页没有“退出应用”，隐私政策另有网页。
class PrivacyGate extends StatefulWidget {
  const PrivacyGate({super.key, required this.child});

  final Widget child;

  @override

```

```

    State<PrivacyGate> createState() => _PrivacyGateState();
}

class _PrivacyGateState extends State<PrivacyGate> {
  @override
  void initState() {
    super.initState();
    if (!kIsWeb) _checkConsent();
  }

  Future<void> _checkConsent() async {
    if (await hasAgreedToPrivacyPolicy() || !mounted) return;
    while (mounted) {
      if (await _ask(_PromptDialog.new)) {
        await recordPrivacyAgreement();
        return;
      }
      if (!mounted) return;
      if (await _ask(_DisagreeDialog.new)) {
        // “再看看”：回到隐私提示。
        continue;
      }
      await SystemNavigator.pop();
      return;
    }
  }

  /// 弹出不能点空白处或返回键关闭的对话框，返回用户是否点了主按钮。
  Future<bool> _ask(Widget Function() dialog) async {
    final result = await showDialog<bool>(
      context: context,
      barrierDismissible: false,
      builder: (_) => PopScope(canPop: false, child: dialog()),
    );
    return result ?? false;
  }

  @override
  Widget build(BuildContext context) => widget.child;
}

class _PromptDialog extends StatelessWidget {
  const _PromptDialog();

  @override
  Widget build(BuildContext context) {
    final l10n = AppLocalizations.of(context);
    return AlertDialog(
      title: Text(l10n.privacyPromptTitle),
      content: SingleChildScrollView(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,

```

```

        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(l10n.privacyPromptBody),
          const SizedBox(height: 8),
          TextButton(
            style: TextButton.styleFrom(padding: EdgeInsets.zero),
            onPressed: () => showPrivacyPolicy(context),
            child: Text(l10n.privacyViewFull),
          ),
        ],
      ),
    ),
    actions: [
      TextButton(
        onPressed: () => Navigator.of(context).pop(false),
        child: Text(l10n.privacyDisagree),
      ),
      FilledButton(
        onPressed: () => Navigator.of(context).pop(true),
        child: Text(l10n.privacyAgree),
      ),
    ],
  );
}
}

```

```

class _DisagreeDialog extends StatelessWidget {
  const _DisagreeDialog();

  @override
  Widget build(BuildContext context) {
    final l10n = AppLocalizations.of(context);
    return AlertDialog(
      title: Text(l10n.privacyDisagreeTitle),
      content: Text(l10n.privacyDisagreeBody),
      actions: [
        TextButton(
          onPressed: () => Navigator.of(context).pop(false),
          child: Text(l10n.privacyExit),
        ),
        FilledButton(
          onPressed: () => Navigator.of(context).pop(true),
          child: Text(l10n.privacyBack),
        ),
      ],
    );
  }
}

```

/// 完整的隐私政策。正文里以“## ”开头的行是小标题。

```

class PrivacyPolicyPage extends StatelessWidget {
  const PrivacyPolicyPage({super.key});

```

```

@override
Widget build(BuildContext context) {
  final l10n = AppLocalizations.of(context);
  final theme = Theme.of(context);
  final lines = [
    ...l10n.privacyPolicyBody.split('\n'),
    ...l10n
      .privacyContact(l10n.privacyDeveloper, l10n.privacyEmail)
      .split('\n'),
  ];
  return Scaffold(
    appBar: AppBar(title: Text(l10n.privacyPolicy)),
    body: SafeArea(
      child: ListView(
        padding: const EdgeInsets.fromLTRB(20, 8, 20, 32),
        children: [
          for (final line in lines)
            if (line.startsWith('## '))
              Padding(
                padding: const EdgeInsets.only(top: 20, bottom: 6),
                child: Text(
                  line.substring(3),
                  style: theme.textTheme.titleMedium,
                ),
              )
            else if (line.isNotEmpty)
              Padding(
                padding: const EdgeInsets.only(bottom: 6),
                child: Text(
                  line,
                  style: theme.textTheme.bodyMedium?.copyWith(height: 1.6),
                ),
              ),
        ],
      ),
    ),
  );
}

import 'package:flutter/material.dart';

import '../l10n/app_localizations.dart';
import '../logic/centering.dart';

/// 固定行高的 strut：不论这一行显示什么字、用到哪种字体，高度都一样。
StrutStyle fixedLine(TextStyle? style) => StrutStyle(
  fontSize: style?.fontSize ?? 14,
  height: 1.4,
  forceStrutHeight: true,
);

```

```

/// 一行文字的固定高度：按字号（含系统字体缩放）算，与实际用到的字体无关。
double fixedLineHeight(BuildContext context, TextStyle? style) =>
    MediaQuery.textScalerOf(context).scale(style?.fontSize ?? 14) * 1.5;

String formatGrade(double g) =>
    g == g.roundToDouble() ? g.toInt().toString() : g.toString();

/// 底部结果栏：左右、上下比例，参考等级，临界提示与免责说明。
class ResultBar extends StatelessWidget {
    const ResultBar({super.key, required this.result});

    final CenteringResult result;

    @override
    Widget build(BuildContext context) {
        final l10n = AppLocalizations.of(context);
        final theme = Theme.of(context);
        final r = result;
        final grade = r.grade;
        final near = r.nearGrade;
        return Material(
            color: theme.colorScheme.surfaceContainerHighest,
            child: SafeArea(
                top: false,
                child: Padding(
                    padding: const EdgeInsets.fromLTRB(16, 12, 16, 8),
                    child: Column(
                        mainAxisAlignment: MainAxisAlignment.min,
                        children: [
                            Row(
                                children: [
                                    _Cell(
                                        label: l10n.leftRight,
                                        value: '${r.lrBig}/${r.lrSmall}',
                                    ),
                                    _Cell(
                                        label: l10n.topBottom,
                                        value: '${r.tbBig}/${r.tbSmall}',
                                    ),
                                    _Cell(
                                        label: l10n.referenceGrade,
                                        value: grade == null
                                            ? l10n.belowStandard
                                            : l10n.gradeValue(formatGrade(grade)),
                                        emphasize: true,
                                    ),
                                ],
                            ),
                            // 临界提示始终占一行：出现或消失时结果栏高度不变，图片不会跳动。
                            Container(
                                height: fixedLineHeight(context, theme.textTheme.bodyMedium),

```

```

        margin: const EdgeInsets.only(top: 6),
        alignment: Alignment.center,
        child: Text(
          r.borderline && near != null
            ? l10n.borderline(formatGrade(near))
            : '',
          key: const Key('borderline'),
          maxLines: 1,
          overflow: TextOverflow.ellipsis,
          // 高度固定：中文和空行在有些系统上用不同字体、行高不同，
          // 提示出现或消失时结果栏会变高变矮，画布跟着跳动。
          strutStyle: fixedLine(theme.textTheme.bodyMedium),
          style: theme.textTheme.bodyMedium?.copyWith(
            color: Colors.orange.shade800,
            fontWeight: FontWeight.w600,
          ),
        ),
      ),
    ),
    // 免责声明缩成一行小字，完整内容在“说明”里。
    Padding(
      padding: const EdgeInsets.only(top: 4),
      child: Text(
        l10n.disclaimer,
        textAlign: TextAlign.center,
        maxLines: 1,
        overflow: TextOverflow.ellipsis,
        style: theme.textTheme.bodySmall?.copyWith(
          fontSize: 10,
          color: theme.colorScheme.onSurfaceVariant,
        ),
      ),
    ),
  ],
),
),
);
}
}

```

```

class _Cell extends StatelessWidget {
  const _Cell({
    required this.label,
    required this.value,
    this.emphasize = false,
  });

  final String label, value;
  final bool emphasize;

  @override
  Widget build(BuildContext context) {

```

```

final theme = Theme.of(context);
return Expanded(
  child: Column(
    children: [
      Text(label, style: theme.textTheme.labelMedium),
      const SizedBox(height: 2),
      SizedBox(
        height: fixedLineHeight(context, theme.textTheme.titleLarge),
        child: FittedBox(
          fit: BoxFit.scaleDown,
          child: Text(
            value,
            // “PSA 9”与“低于 PSA 3”字体不同，同样固定高度。
            strutStyle: fixedLine(theme.textTheme.titleLarge),
            style: theme.textTheme.titleLarge?.copyWith(
              fontWeight: FontWeight.w700,
              color: emphasize ? theme.colorScheme.primary : null,
            ),
          ),
        ),
      ],
    ),
  ),
);
}

```